

المحاضرة 09

المصفوفات (1) : Arrays

د. سهيل الحمود

د. أسامه ناصر

2025-2026

المصفوفات في لغة ++C

What is an Array (ما هي المصفوفة؟)

- المصفوفة هي مجموعة من العناصر من نفس نوع البيانات
- يتم تخزين العناصر في الذاكرة بشكل متتابع
- حجم المصفوفة ثابت

```
int marks[5];
```

لماذا نحتاج المصفوفات؟

- نحتاج إلى المصفوفات عند التعامل مع عدد كبير من القيم المتشابهة
- استخدام عدد كبير من المتغيرات هو حل غير عملي
- استخدام متغير لكل قيمة يجعل البرنامج: صعب الكتابة و التعديل

```
int mark1, mark2, mark3, mark4, mark5; // ❌
```

- الأفضل في هذه الحالة استخدام المصفوفات Arrays:
- هذا يجعل الكود أبسط و أسهل في القراءة و الصيانة

```
int marks[5]; // ✅
```

Declaration & Initialization (التصريح والتهيئة)

• بعدة طرق:

```
int a[5];  
int b[5] = {1, 2, 3, 4, 5};  
int c[5] = {1, 2};  
int d[] = {1, 2, 3, 4, 5, 6};
```

- المصفوفة غير المهيأة تحتوي قيما عشوائية
- التهيئة الجزئية تجعل القيم المتبقية مساوية لـ 0
- عدد عناصر المصفوفة d هي 6 عناصر

Indexing (الفهرسة)

- يبدأ الفهرس من 0
- قيمة آخر فهرس = الحجم - 1

مثال:

```
char a[] = {'A', 'B', 'C', 'D', 'E'};
```

شرح:

index :	0	1	2	3	4
values:	_A	B	C	D	E

Accessing Array Elements (الوصول إلى عناصر المصفوفة)

باستخدام الفهرس:

```
b[0] = 10; // إسناد قيمة إلى عنصر في مصفوفة  
cout << b[2]; // قراءة قيمة عنصر من مصفوفة
```

⚠ محاولة قراءة عنصر مصفوفة بفهرس قيمته خارج حدود المصفوفة ينتج خطأ

```
int b[5];  
b[5] = 100; // خطأ
```

آخر عنصر في المصفوفة السابقة له قيمة فهرس تساوي 4 أو (حجم المصفوفة-1)

Input Using Loops (إدخال البيانات باستخدام الحلقات)

تهيئة المصفوفة بعدد عناصر ثابت ثم استخدام حلقة لها نفس عدد عناصر المصفوفة

```
cout << "Enter the size of array: ";  
int n;  
cin >> n;  
  
int arr[n];  
  
for (int i = 0; i < n; i++) {  
    cin >> arr[i];  
}
```

Output Using Loops (إخراج البيانات باستخدام الحلقات)

```
int arr[5] = {11, 22, 33, 44, 55};  
  
for (int i = 0; i < 5; i++) {  
    cout << arr[i] << " ";  
}
```

Basic Array Operations (العمليات الأساسية على المصفوفات)

- استعراض عناصر المصفوفة
- حساب المجموع
- حساب المتوسط
- إيجاد أكبر قيمة
- إيجاد أصغر قيمة

مثال (إيجاد أكبر عنصر):

```
int max = arr[0];
for (int i = 1; i < n; i++) {
    if (arr[i] > max)
        max = arr[i];
}
//now max has the maximum value of the array
```

Range-Based For Loop

المرور على عناصر المصفوفة باستخدام حلقة for المعتمدة على النطاق

الصيغة العامة

```
for (range_declaration : range_expression) {  
    // loop body  
}
```

مثال على المصفوفة:

```
int arr[] = {11, 22, 33, 44, 55};  
  
for (int x : arr) {  
    cout << x << " ";  
}
```

Passing Arrays to Functions (تمرير المصفوفة إلى الدوال)

```
void printArray(int arr[], int n) {  
    for (int i = 0; i < n; i++)  
        cout << arr[i] << " ";  
}
```

- لا يمكن استخدام التابع sizeof على المصفوفة التي تم تمريرها كمعامل ضمن الدالة
- لذا يجب تمرير حجم المصفوفة
- يتم تمرير المصفوفة كمرجع (Call by Reference)

Passing Elements of an Array to Functions

(تمرير عناصر من مصفوفة إلى دالة)

يتم تمرير عناصر المصفوفة كقيم (Call by Value)

```
void process(int a){
    a++;
    cout << "inside function a= " << a << endl;
}

int main() {
    int arr[] = {11, 22, 33};
    process(arr[0]);
    cout << arr[0] << endl;
}
```

تمرين 1: حساب المتوسط لمصفوفة عددية

```
double average(double arr[], int size) {
    double sum = 0;
    for (int i = 0; i < size; i++){
        sum += arr[i];
    }
    return sum / size;
}

int main(){
    double arr[4] = {40, 50, 60, 70};
    cout << average(arr, 4);
    return 0;
}
```

تمرين 2- البحث عن وجود عنصر في مصفوفة Linear Search

تحديد إذا كانت المصفوفة تحوي عنصرا ما

تحقيق التمرين باستخدام دالة function

```
bool contains(  
    int[] arr,  
    int size,  
    int key){  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == key)  
            return true;  
    }  
    return false;  
}
```

خروج من الحلقة عند إيجاد العنصر

```
int key;  
bool found = false;  
for (int i = 0; i < n; i++) {  
    if (arr[i] == key) {  
        found = true;  
        break;  
    }  
}  
cout << "found: " << found << endl;
```

تمرين 3- البحث عن وجود عنصر في مصفوفة Linear Search

باستخدام الشكل الآخر للحلقة For, الحلقة المعتمدة على المجال

خروج من الحلقة عند إيجاد العنصر

تحقيق التمرين باستخدام دالة function

```
bool contains(  
    int[] arr,  
    int size,  
    int key){  
    for (int v: arr) {  
        if (v == key)  
            return true;  
    }  
    return false;  
}
```

```
int key;  
bool found = false;  
for (int v: arr) {  
    if (v == key) {  
        found = true;  
        break;  
    }  
}  
cout << "found: " << found << endl;
```

تمرين 4- البحث عن مكان وجود عنصر في مصفوفة Linear Search

- البحث عن عنصر في مصفوفة و إرجاع موضعه في حال كان موجودا
- يقوم بإرجاع الفهرس لأول موضع يجد فيه العنصر
- في حال لم نجد العنصر ترجع الدالة قيمة -1
- لاحظ استدعاء return و الخروج مبكرا من الدالة عند إيجاد الجواب

```
int findIndex(int arr[], int size, int key) {  
    for (int i = 0; i < size; i++) {  
        if (arr[i] == key)  
            return i;    // element found  
    }  
    return -1;           // element not found  
}
```

تمرين 5- عد الأعداد الزوجية والفردية في مصفوفة

```
int arr[] = { 1, 4, 6, 7, 3, 11, 15, 6 };

const int SIZE = sizeof(arr) / sizeof(arr[0]);

int even = 0, odd = 0;
for (int i = 0; i < SIZE; i++) {
    if (arr[i] % 2 == 0)
        even++;
    else
        odd++;
}

cout << "Even numbers = " << even << endl;
cout << "Odd numbers = " << odd << endl;
```

تمرين 6- عد الأعداد الزوجية والفردية في مصفوفة

طريقة مختصرة

```
int arr[] = {1, 4, 6, 7, 3, 11, 15, 6};  
  
int result[] = {0, 0};  
  
for (int v: arr)  
    result[v % 2]++;  
  
cout << "Even numbers = " << result[0] << endl;  
cout << "Odd numbers = " << result[1] << endl;
```

تمرين 6 - عكس مصفوفة طريقة أولى: حلقة for

```
void reverse(int arr[], int size) {  
    int mid = size / 2;  
    for (int i = 0; i < mid; i++) {  
        const int temp = arr[i];  
        arr[i] = arr[size - i - 1];  
        arr[size - i - 1] = temp;  
    }  
}
```

تمرين 7 - عكس مصفوفة

طريقة ثانية: حلقة while

```
void reverse(int arr[], int size) {  
  
    int start = 0;  
    int end = size - 1;  
  
    while (start < end) {  
        swap(arr[start], arr[end]);  
        start++;  
        end--;  
    }  
}
```

تمرين 8 - عكس مصفوفة

طريقة ثالثة: باستخدام الاستدعاء العودي Recursion

```
void reverse(int arr[], int start, int end) {  
    if (start ≥ end) return;  
    std::swap(arr[start], arr[end]);  
    reverse(arr, start + 1, end - 1);  
}  
  
int main() {  
    int arr[] = {1, 4, 6, 7, 3, 11, 15, 6, 55};  
  
    reverse(arr, 0, 8);  
  
    for (int v: arr) cout << v << " ";  
}
```